

Classes referenciando outras Classes

1 - Classes referenciando outras Classes

No Tutorial 1, ilustramos as implementações das classes sem relacionamentos: [AgênciaPublicidade](#) e [Veículo](#). Neste tutorial, vamos ilustrar as implementações das entidades que referenciam objetos de outras entidades: (a) a entidade de relacionamento múltiplo [Montadora](#); e (b) a entidade de associação [Lançamento](#).

1.1 - Entidade de Relacionamento Múltiplo

A seguir, a implementação do módulo [montadora](#) do diretório [entidades](#).

```
montadoras = {}

def get_montadoras(): return montadoras

def inserir_montadora(montadora):
    nome_montadora = montadora.nome
    if nome_montadora not in montadoras.keys():
        montadoras[nome_montadora] = montadora
        return True
    else:
        print('Montadora ' + nome_montadora + ' já tem cadastro')
        return False

class Montadora:
    def __init__(self, nome, país_origem, sede_maior_fábrica_brasil):
        self.nome = nome
        self.país_origem = país_origem
        self.sede_maior_fábrica_brasil = sede_maior_fábrica_brasil
        self.veículos = {}

    def __str__(self):
        formato = '{ } {<20} { } {<14} { } {<17} { }'
        montadora_formatada = formato.format("|", self.nome, "|", self.país_origem, "|", self.sede_maior_fábrica_brasil, "|")
        return montadora_formatada

    def inserir_veículo(self, veículo):
        marca_modelo_veículo = veículo.marca_modelo
        if marca_modelo_veículo not in self.veículos.keys(): self.veículos[marca_modelo_veículo] = veículo
        else: print('Veículo ' + marca_modelo_veículo + ' já tem cadastro na Montadora')
```

A estrutura de dados mais versátil para armazenar um conjunto de objetos de uma classe associada a um único identificador é o dicionário, que permite armazenar cada objeto dessa classe associado à sua chave. Desta forma, poderemos recuperar o objeto a partir do valor da sua chave, dado que ela é única (não se repete) no conjunto de objetos armazenados. Mas fique atento: as chaves de um dicionário devem ser definidas como strings.

O conjunto [montadoras](#) é inicializado com um dicionário vazio (`{ }`). A função [get_montadoras](#) retorna o conjunto de [montadoras](#). É importante diferenciar a utilização de listas e dicionários. A função [get_montadoras](#) retorna um dicionário. Diferentemente de uma lista, um dicionário é um conjunto que contém para cada elemento do conjunto, o par: chave (string) e valor (objeto). Para obter os objetos de um dicionário, você precisará utilizar a função [values](#), que retorna os objetos armazenados no dicionário: `montadoras = get_montadoras().values()`.

A função `inserir_montadora` recebe um objeto `montadora`, verifica se sua chave ainda não foi inserida no dicionário, antes de associar o objeto `montadora` à sua chave (`nome`) no dicionário. Para comparar a chave do objeto com as chaves armazenadas no dicionário, é utilizado o método `keys` que retorna as chaves do dicionário. Para inserir um novo par chave (`nome_montadora`) e valor (`montadora`) no dicionário, é utilizado o comando: `montadoras[nome_montadora] = montadora`.

Observe que as classes `Veículo` e `AgênciaPublicidade` de publicidade, tem um único identificador, respectivamente: (a) `marca_modelo` para `Veículo`; e (b) `nome` para `AgênciaPublicidade`. Desta forma, neste tutorial vamos alterar os conjuntos utilizados no Tutorial 1 para armazenar os objetos dessas classes, de listas para dicionários.

No projeto de referência, que foi especificado no Tutorial Auxiliar A, que a classe `Montadora` tem um relacionamento múltiplo com a entidade `Veículo`. O relacionamento múltiplo pode ser [1:n] ou [n:n], em função do projeto. No projeto de referência escolhido é utilizado um relacionamento [1:n], objetos da classe `Veículo` são associados a um único objeto da classe `Montadora`, dado que uma marca modelo de veículo é produzido por uma única montadora. Mas existem projetos com relacionamento [n:n], nos quais objetos podem ser vinculados a mais de um objeto da classe principal do relacionamento múltiplo, com por exemplo, atores que podem participar de vários filmes.

Em função do fato de que veículos são produzidos por uma única montadora, os objetos da classe `Veículo` estão sendo armazenados somente em objetos da classe `Montadora`, no dicionário `veículos`. Adicionalmente é implementado, na classe `Montadora`, o método `inserir_veículo`, que recebe o objeto de um veículo, e verifica se sua chave não pertence ao dicionário `veículos` antes de inserir o objeto no dicionário.

Além de alterar o conjunto de objetos da classe `AgênciaPublicidade`, de lista para dicionário no módulo `agência_publicidade`, serão necessário implementar o método `inserir_agência_publicidade`, que verifica se o nome da agência de publicidade (chave) não pertence ao dicionário `agências_publicidade` antes de inserir o objeto no dicionário.

```
agências_publicidade = {}

def inserir_agência_publicidade(agência_publicidade):
    nome_agência_publicidade = agência_publicidade.nome
    if nome_agência_publicidade not in agências_publicidade.keys():
        agências_publicidade[nome_agência_publicidade] = agência_publicidade
        return True
    else:
        print('Agência de Publicidade ' + nome_agência_publicidade + ' já tem cadastro')
        return False
```

Como veremos na próxima subseção, nesta etapa do projeto a filtragem será realizada somente com os objetos da entidade de Associação `Lançamento`. Consequentemente, os métodos `selecionar_veículos` e `selecionar_agências_publicidade` das classes `Veículo` e `AgênciaPublicidade`, foram removidos na segunda etapa do projeto.

Antes de finalizar essa seção, é importante comentarmos como proceder em projetos nos quais objetos de uma entidade secundária do relacionamento múltiplo podem pertencer a mais de uma objeto da entidade principal do relacionamento múltiplo. Esse final de subseção não se relaciona com o projeto de referência que estamos desenvolvendo. Tem apenas o objetivo de orientar alunos que tenham projetos com relacionamento múltiplo diferente do que ocorre no projeto que estamos desenvolvendo neste tutorial.

Embora a entidade **Filme** agregue a entidade **Ator**, um ator pode participar de mais de um filme. No módulo **ator**, serão definidos o dicionário **atores** e o método **inserir_ator(ator)**, dado um dado objeto **ator** poderá estar relacionado com mais de um objeto **filme**.

A classe **Filme**, do módulo **filme**, vai incorporar: (a) o dado **atores**, definido inicialmente como um dicionário vazio; e o método **inserir_atores(nomes_atores)**, que vai receber uma lista de nomes de atores e preencher o dicionário **atores** com os objetos obtidos a partir dos nomes dos atores. Esse procedimento ocorre, porque os atores, que podem participar de mais de um filme, serão todos previamente inseridos no conjunto **filmes** do módulo **filme**, antes de serem inseridos no dicionário **atores** da classe **Filme**. Como não faz sentido criar dois objetos da classe **Ator** com o mesmo conteúdo, na classe **Filme** do módulo **filme** serão repassados somente os nomes dos atores, para referenciar os objetos previamente cadastrados no dicionário **atores** do módulo **ator**.

```
def inserir_atores(self, nomes_atores):
    for nome_ator in nomes_atores:
        if nome_ator in get_atores().keys():
            self.atores[nome_ator] = get_atores()[nome_ator]
        else:
            print('Ator ' + nome_ator + ' não tem cadastro')
```

1.2 - Entidade de Associação

Nesta subseção ilustrar a implementação do módulo **lançamento** do diretório **entidades**, que irá conter a classe **Lançamento**. Um lançamento é realizado por uma agência de publicidade, a pedido de uma montadora (para todos os veículos que a montadora decide lançar), em uma determinada data. A implementação do módulo **lançamento** é ilustrada na página seguinte.

Por não ter uma chave única, os objetos da classe **Lançamento** são armazenados em uma lista. No lugar da função **inserir_lançamento** é implementada a função **criar_lançamento(nome_montadora, nome_agência_publicidade, data)**. Esse método obtém os objetos referenciados a partir de suas chaves, com exceção do objeto da classe **Data** que já é passado como argumento na chamada de **criar_lançamento**. Após obter os objetos necessários um objeto é criado com a execução do construtor da classe **Lançamento**, e o objeto criado é apendado ao conjunto **lançamentos**, após ser verificado que ainda não pertence ao conjunto. Caso não obtenha alguns dos objetos necessários, é impressa uma mensagem de erro e a função retorna sem realizar a criação do objeto da classe **Lançamento**.

```
from entidades.agência_publicidade import get_agências_publicidade
from entidades.montadora import get_montadoras

lançamentos = []

def get_lançamentos(): return lançamentos
```

```

def inserir_lançamento(lançamento):
    if lançamento not in lançamentos: lançamentos.append(lançamento)
    else: print('Lançamento de Veículo já tem cadastro --- ' + str(lançamento))

def criar_lançamento(nome_montadora, nome_agência_publicidade, data):
    montadora = get_montadoras()[nome_montadora]
    if montadora is None:
        print('Montadora ' + nome_montadora + ' não cadastrada')
        return
    agência_publicidade = get_agências_publicidade()[nome_agência_publicidade]
    if agência_publicidade is None:
        print('Agência de Publicidade ' + nome_agência_publicidade + ' não cadastrada')
        return
    lançamento = Lançamento(montadora, agência_publicidade, data)
    if lançamento not in lançamentos: lançamentos.append(lançamento)
    else: print('Lançamento de Veículo já tem cadastro --- ' + str(lançamento))

def selecionar_lançamentos(data_mínima_lançamento=None, potência_mínima_veículo=None,
    ranking_máximo_avaliação_agência_publicidade=None, país_origem_montadora=None):
    filtros = '\nFiltros -- '
    if data_mínima_lançamento is not None: filtros += 'data mínima de lançamento: ' + str(data_mínima_lançamento)
    if potência_mínima_veículo is not None: filtros += ' - potência mínima do veículo: ' + str(potência_mínima_veículo)
    if ranking_máximo_avaliação_agência_publicidade is not None:
        filtros += ('\n - ranking máximo de avaliação da agência de publicidade: '
            + str(ranking_máximo_avaliação_agência_publicidade))
    if país_origem_montadora is not None: filtros += (' - país de origem da montadora: ' + país_origem_montadora)
    lançamentos_selecionados = []
    for lançamento in lançamentos:
        if data_mínima_lançamento is not None and lançamento.data < data_mínima_lançamento: continue
        excluir_lançamento = False
        for veículo in lançamento.montadora.veículos.values():
            if potência_mínima_veículo is not None and veículo.potência < potência_mínima_veículo:
                excluir_lançamento = True
                break
        if excluir_lançamento: continue
        if (ranking_máximo_avaliação_agência_publicidade is not None
            and lançamento.agência_publicidade.ranking_avaliação > ranking_máximo_avaliação_agência_publicidade): continue
        if país_origem_montadora is not None and lançamento.montadora.país_origem != país_origem_montadora: continue
        lançamentos_selecionados.append(lançamento)
    return filtros, lançamentos_selecionados

class Lançamento:
    def __init__(self, montadora, agência_publicidade, data):
        self.montadora = montadora
        self.agência_publicidade = agência_publicidade
        self.data = data

    def __str__(self):
        formato = '{:<20} {} {:<10} {} {:<10} {}'
        lançamento_formatado = formato.format(' ', self.montadora.nome, ' ', self.agência_publicidade.nome, ' ', str(self.data), ' ')
        return lançamento_formatado

    def str_atributos_veículos(self):
        atributos_veículos_str = ''
        for índice, veículo in enumerate(self.montadora.veículos.values()):
            if (índice > 0): atributos_veículos_str += ' - '
            atributos_veículos_str += f'{veículo.potência:3d}' + ' cv'
        return atributos_veículos_str

    def str_filtro(self):
        formato = '{:>2} {} {:<14} {} {:<15} {}'
        filtro_formatado = formato.format(str(self.agência_publicidade.ranking_avaliação),
            ' ', self.montadora.país_origem, ' ', self.str_atributos_veículos(), ' ')
        return self.__str__() + filtro_formatado

```

Nesta etapa do projeto, a filtragem é implementada somente para objetos da classe de Associação [Lançamento](#), utilizando um filtro oriundo de todas as entidades envolvidas. Observe que o método [selecionar_lançamentos](#) recebe como argumentos filtros relacionados com um atributo de cada entidade do projeto.

Os nomes dos filtros devem expressar completamente o seu significado, incorporando a classe do atributo ao qual está relacionado e se o teste envolve um limite inferior (valor mínimo) ou superior (valor máximo), quando for o caso. Limites inferiores ou superiores são utilizados no projeto de referência sempre que a comparação é feita com um atributo inteiro ou flutuante.

Observe que o nome do filtro [ranking_máximo_avaliação_agência_publicidade](#) expressa perfeitamente: o atributo original ([ranking_avaliação](#)), a classe de origem do atributo ([agência_publicidade](#)) e o limite utilizado (valor máximo). Essa regra de formação de nomes de filtros é muito importante para a legibilidade do código e deve ser utilizada por todos os alunos da disciplina.

O loop de teste dos objetos selecionados itera nos objetos da lista [lançamentos](#). Quando for necessário comparar um filtro oriundo de outra classe com o respectivo atributo da classe, é necessário obter o atributo a partir do objeto lançamento.

Por exemplo, para obter o atributo ranking de avaliação do agência de publicidade que está sendo lançado para comparar com o filtro [ranking_máximo_avaliação_agência_publicidade](#) é necessário obtê-lo a partir do objeto lançamento: [lançamento.agência_publicidade.rating_avaliação](#).

Observe, no entanto, que os veículos não são referenciados diretamente por um objeto da entidade [Lançamento](#), mas sim a partir do objeto da entidade [Montadora](#) obtido a partir do objeto da entidade [Lançamento](#). Adicionalmente, em função do relacionamento múltiplo, vários veículos podem ser associados a um única montadora, no dicionário local ([self.veículos](#)) da entidade [Montadora](#). Neste caso, será necessário implementar um loop para obter o atributo potência de cada veículo da montadora, e somente selecionar o lançamento no qual todos os veículos da montadora atendam o filtro [potência_mínima_veículo](#).

Além dos atributos do objeto da classe lançamento, os objetos selecionados devem mostrar também os atributos das outras classes que deverão ser comparados com os valores solicitados pelos filtros passados como argumentos para a função [selecionar_lançamentos](#). Para atender essa necessidade é implementado na classe [Lançamento](#) o método [str_filtro](#), que retorna um string no qual apenas aos atributos do lançamento, os atributos necessários das outras entidades.

2 - Controle do Projeto

No diretório [controle](#) é implementado o módulo [projeto](#), utilizado para controlar toda a aplicação. A função [cadastrar_agências_publicidade](#) foi omitida, porque é a mesma ilustrada no Tutorial 1.

```
from util.gerais import imprimir_objetos, imprimir_objeto, imprimir_objetos_internos, imprimir_objetos_associacao_filtros
from util.data import Data
from entidades.agência_publicidade import inserir_agência_publicidade, AgênciaPublicidade, get_agências_publicidade
from entidades.veículo import Veículo
from entidades.montadora import inserir_montadora, Montadora, get_montadoras
from entidades.lançamento import criar_lançamento, get_lançamentos, selecionar_lançamentos
```

Inicialmente, são importadas as funções e as classes de outro módulos que serão utilizados no módulo `projeto`. Na página seguinte é ilustrada a função `cadastrar_montadoras`. Nesta função, para cada objeto da classe `Montadora`: (a) é criado o objeto utilizando o construtor da classe; (b) inserido o objeto no conjunto `montadoras` (do módulo `montadora`) com a função `inserir_montadora`; e (c) são inseridos cada um dos veículos da montadora no objeto `montadora` com o método `inserir_veículo`.

Na função `cadastrar_lançamentos`, os objetos da classe `Lançamento` são criados e inseridos no conjunto `lançamentos` (do módulo `lançamento`) com o método `criar_lançamento`.

No corpo principal do programa (que comentaremos em outra página adiante), a impressão dos veículos de cada montadora será realizada imediatamente após a impressão de cada objeto da montadora. Para facilitar a visualização necessária para o alinhamento na formatação de objetos das montadoras, separados de objetos de veículos, é implementada a função auxiliar `imprimir_somente_para_alinhar_formatação`.

```
def cadastrar_montadoras():
    montadora = Montadora(nome='Hyundai', país_origem='Coréia do Sul', sede_maior_fábrica_brasil='Piracicaba - SP')
    inserir_montadora(montadora)
    montadora.inserir_veículo(Veículo(marca_modelo='Hyundai HB20', alimentação='flex', potência=80, importado=False))
    montadora = Montadora('GM', 'Estados Unidos', 'São Caetano - SP')
    inserir_montadora(montadora)
    montadora.inserir_veículo(Veículo('Chevrolet Onix Plus', 'flex', 116, False))
    montadora = Montadora('Fiat', 'Itália', 'Betim - MG')
    inserir_montadora(montadora)
    montadora.inserir_veículo(Veículo('Fiat Strada', 'diesel', 130, False))
    montadora.inserir_veículo(Veículo('Fiat Toro', 'diesel', 185, False))
    montadora = Montadora('BYD', 'China', 'Camaçari - BA')
    inserir_montadora(montadora)
    montadora.inserir_veículo(Veículo('BYD Dolphin', 'elétrico', 95, True))
    montadora = Montadora('Volvo', 'Suécia', 'exterior')
    inserir_montadora(montadora)
    montadora.inserir_veículo(Veículo('Volvo XC40', 'elétrico', 231, True))
    montadora = Montadora('Volvo Caminhões', 'Suécia', 'Curitiba - PR')
    inserir_montadora(montadora)
    montadora.inserir_veículo(Veículo('Volvo FH 540', 'diesel', 540, False))
    montadora.inserir_veículo(Veículo('Volvo FH 460', 'diesel', 460, False))
    montadora = Montadora('Volkswagen Caminhões', 'Alemanha', 'Resende - RJ')
    inserir_montadora(montadora)
    montadora.inserir_veículo(Veículo('VW Delivery 11180', 'diesel', 175, False))
    montadora = Montadora('DAF', 'Holanda', 'Ponta Grossa - PR')
    inserir_montadora(montadora)
    montadora.inserir_veículo(Veículo('DAF XF 530', 'diesel', 530, False))

def cadastrar_lançamentos():
    criar_lançamento(nome_montadora='Hyundai', nome_agência_publicidade='Africa', data=Data(dia=1, mês=9, ano=2012))
    criar_lançamento('GM', 'McCann', Data(12,9,2019))
    criar_lançamento('Fiat', 'Galeria', Data(1,9,1998))
    criar_lançamento('Fiat', 'Galeria', Data(1,9,2016))
    criar_lançamento('BYD', 'Artplan', Data(28,6,2023))
    criar_lançamento('Volvo', 'Artplan', Data(15,4,2018))
    criar_lançamento('Volvo Caminhões', 'BETC Havas', Data(1,3,2014))
    criar_lançamento('Volkswagen Caminhões', 'Galeria', Data(23,1,2018))
    criar_lançamento('DAF', 'Galeria', Data(18,8,2020))
    criar_lançamento('Volvo Caminhões', 'BETC Havas', Data(17,4,2014))
```


Na página seguinte, é ilustrado o corpo principal do projeto composto de:

- cadastro ([cadastrar_agências_publicidade](#)) dos objetos no dicionário [agências_publicidade](#) (módulo [agência_publicidade](#)) e impressão ([imprimir_objetos](#)) desses objetos;
- cadastro ([cadastrar_montadoras](#)) dos objetos no dicionário [montadoras](#) (módulo [montadora](#)), juntamente com os objetos dos veículos vinculados a cada montadora, e impressão ([imprimir_objetos](#)) do conjunto de montadoras;
- loop de impressão das montadoras ([imprimir_objeto](#)) e seus respectivos veículos ([imprimir_objetos_internos](#));
- cadastro ([cadastrar_lançamentos](#)) dos objetos no dicionário [lançamentos](#) (módulo [lançamento](#)) e impressão ([imprimir_objetos](#)) desses objetos;
- filtragem dos lançamentos ([selecionar_lançamentos](#)) e impressão dos filtros utilizados e dos objetos filtrados ([imprimir_objetos_associiação_filtros](#)); inicialmente sem filtros e então acrescentando um a um os valores dos filtros previstos na função de filtragem.

```
if __name__ == '__main__':
    cadastrar_agências_publicidade()
    imprimir_objetos(cabeçalho='\nAgências de Publicidades : nome - país de origem - ranking de avaliação',
                    objetos=get_agências_publicidade().values())
    cadastrar_montadoras()
    imprimir_objetos(cabeçalho='\nAMontadoras : nome - país_origem - sede_maior_fábrica_brasil',
                    objetos=get_montadoras().values())
    print('\nMontadoras : nome - país de origem - sede da maior fábrica no Brasil')
    print(' - Veículos : marca modelo - alimentação - potência - importado')
    for indice, montadora in enumerate(get_montadoras().values()):
        imprimir_objeto(indice=indice, objeto_str=str(montadora))
        imprimir_objetos_internos(montadora.veículos.values())
    cadastrar_lançamentos()
    cabeçalho_lançamento = ('Lançamentos : nome da montadora - marca modelo do veículo - nome da agência de publicidade'
                           + ' - data do lançamento')
    imprimir_objetos('\n' + cabeçalho_lançamento, get_lançamentos())
    filtros, lançamentos_selecionados = selecionar_lançamentos()
    cabeçalho_lançamento_filtros = (cabeçalho_lançamento
                                   + '\n -- ranking de avaliação da agência de publicidade - país de origem da montadora - potências dos veículos')
    imprimir_objetos_associiação_filtros(cabeçalho_lançamento_filtros, lançamentos_selecionados, filtros)
    filtros, lançamentos_selecionados = selecionar_lançamentos(data_mínima_lançamento=Data(dia=1, mês=1, ano=2012))
    imprimir_objetos_associiação_filtros(cabeçalho_lançamento_filtros, lançamentos_selecionados, filtros)
    filtros, lançamentos_selecionados = selecionar_lançamentos(Data(1, 1, 2012), potência_mínima_veículo=150)
    imprimir_objetos_associiação_filtros(cabeçalho_lançamento_filtros, lançamentos_selecionados, filtros)
    filtros, lançamentos_selecionados = selecionar_lançamentos(Data(1, 1, 2012), 150,
                                                                ranking_máximo_avaliação_agência_publicidade=2)
    imprimir_objetos_associiação_filtros(cabeçalho_lançamento_filtros, lançamentos_selecionados, filtros)
    filtros, lançamentos_selecionados = selecionar_lançamentos(Data(1, 1, 2012), 150, 2, país_origem_montadora='Suécia')
    imprimir_objetos_associiação_filtros(cabeçalho_lançamento_filtros, lançamentos_selecionados, filtros)
```

A função [imprimir_objetos_associiação_filtros](#) é implementada no módulo [gerais](#) do diretório [util](#).

No loop de impressão das montadoras é chamada a função [imprimir_objetos_internos](#) para imprimir os veículos de cada montadora, sem mostrar o índice de cada veículo.

```
def imprimir_objetos_associiação_filtros(cabeçalho, objetos, filtros=None):
    if filtros is not None: print(filtros)
    print(cabeçalho)
    for indice, objeto in enumerate(objetos):
        imprimir_objeto(indice, objeto.str_filtro())

def imprimir_objetos_internos(objetos):
    for objeto in objetos: print(' - ' + str(objeto))
```

Finalmente é ilustrada da saída gerada pela execução do projeto.

Agências de Publicidades : nome - país de origem - ranking de avaliação

```
1 - | BETC Havas | França | 1 |
2 - | Galeria | Brasil | 2 |
3 - | Artplan | Brasil | 3 |
4 - | McCann | Estados Unidos | 4 |
5 - | Africa | Estados Unidos | 5 |
```

Montadoras : nome - país_origem - sede_maior_fábrica_brasil

```
1 - | Hyundai | Coréia do Sul | Piracicaba - SP |
2 - | GM | Estados Unidos | São Caetano - SP |
3 - | Fiat | Itália | Betim - MG |
4 - | BYD | China | Camaçari - BA |
5 - | Volvo | Suécia | exterior |
6 - | Volvo Caminhões | Suécia | Curitiba - PR |
7 - | Volkswagen Caminhões | Alemanha | Resende - RJ |
8 - | DAF | Holanda | Ponta Grossa - PR |
```

Montadoras : nome - país de origem - sede da maior fábrica no Brasil

- Veículos : marca modelo - alimentação - potência - importado

```
1 - | Hyundai | Coréia do Sul | Piracicaba - SP |
  - | Hyundai HB20 | flex | 80 cv |
2 - | GM | Estados Unidos | São Caetano - SP |
  - | Chevrolet Onix Plus | flex | 116 cv |
3 - | Fiat | Itália | Betim - MG |
  - | Fiat Strada | diesel | 130 cv |
  - | Fiat Toro | diesel | 185 cv |
4 - | BYD | China | Camaçari - BA |
  - | BYD Dolphin | elétrico | 95 cv | importado |
5 - | Volvo | Suécia | exterior |
  - | Volvo XC40 | elétrico | 231 cv | importado |
6 - | Volvo Caminhões | Suécia | Curitiba - PR |
  - | Volvo FH 540 | diesel | 540 cv |
  - | Volvo FH 460 | diesel | 460 cv |
7 - | Volkswagen Caminhões | Alemanha | Resende - RJ |
  - | VW Delivery 11180 | diesel | 175 cv |
8 - | DAF | Holanda | Ponta Grossa - PR |
  - | DAF XF 530 | diesel | 530 cv |
```

Lançamentos : nome da montadora - nome da agência de publicidade - data do lançamento

```
1 - | Hyundai | Africa | 01/09/2012 |
2 - | GM | McCann | 12/09/2019 |
3 - | Fiat | Galeria | 01/09/1998 |
4 - | Fiat | Galeria | 01/09/2016 |
5 - | BYD | Artplan | 28/06/2023 |
6 - | Volvo | Artplan | 15/04/2018 |
7 - | Volvo Caminhões | BETC Havas | 01/03/2014 |
8 - | Volkswagen Caminhões | Galeria | 23/01/2018 |
9 - | DAF | Galeria | 18/08/2020 |
10 - | Volvo Caminhões | BETC Havas | 17/04/2014 |
```


Filtros --

Lançamentos : nome da montadora - nome da agência de publicidade - data do lançamento

-- ranking de avaliação da agência de publicidade - país de origem da montadora - potências dos veículos

1 -	Hyundai	Africa	01/09/2012	5	Coréia do Sul	80 cv	
2 -	GM	McCann	12/09/2019	4	Estados Unidos	116 cv	
3 -	Fiat	Galeria	01/09/1998	2	Itália	130 cv - 185 cv	
4 -	Fiat	Galeria	01/09/2016	2	Itália	130 cv - 185 cv	
5 -	BYD	Artplan	28/06/2023	3	China	95 cv	
6 -	Volvo	Artplan	15/04/2018	3	Suécia	231 cv	
7 -	Volvo Caminhões	BETC Havas	01/03/2014	1	Suécia	540 cv - 460 cv	
8 -	Volkswagen Caminhões	Galeria	23/01/2018	2	Alemanha	175 cv	
9 -	DAF	Galeria	18/08/2020	2	Holanda	530 cv	
10 -	Volvo Caminhões	BETC Havas	17/04/2014	1	Suécia	540 cv - 460 cv	

Filtros -- data mínima de lançamento: 01/01/2012

Lançamentos : nome da montadora - nome da agência de publicidade - data do lançamento

-- ranking de avaliação da agência de publicidade - país de origem da montadora - potências dos veículos

1 -	Hyundai	Africa	01/09/2012	5	Coréia do Sul	80 cv	
2 -	GM	McCann	12/09/2019	4	Estados Unidos	116 cv	
3 -	Fiat	Galeria	01/09/2016	2	Itália	130 cv - 185 cv	
4 -	BYD	Artplan	28/06/2023	3	China	95 cv	
5 -	Volvo	Artplan	15/04/2018	3	Suécia	231 cv	
6 -	Volvo Caminhões	BETC Havas	01/03/2014	1	Suécia	540 cv - 460 cv	
7 -	Volkswagen Caminhões	Galeria	23/01/2018	2	Alemanha	175 cv	
8 -	DAF	Galeria	18/08/2020	2	Holanda	530 cv	
9 -	Volvo Caminhões	BETC Havas	17/04/2014	1	Suécia	540 cv - 460 cv	

Filtros -- data mínima de lançamento: 01/01/2012 - potência mínima do veículo: 150

Lançamentos : nome da montadora - nome da agência de publicidade - data do lançamento

-- ranking de avaliação da agência de publicidade - país de origem da montadora - potências dos veículos

1 -	Volvo	Artplan	15/04/2018	3	Suécia	231 cv	
2 -	Volvo Caminhões	BETC Havas	01/03/2014	1	Suécia	540 cv - 460 cv	
3 -	Volkswagen Caminhões	Galeria	23/01/2018	2	Alemanha	175 cv	
4 -	DAF	Galeria	18/08/2020	2	Holanda	530 cv	
5 -	Volvo Caminhões	BETC Havas	17/04/2014	1	Suécia	540 cv - 460 cv	

Filtros -- data mínima de lançamento: 01/01/2012 - potência mínima do veículo: 150

- ranking máximo de avaliação da agência de publicidade: 2

Lançamentos : nome da montadora - nome da agência de publicidade - data do lançamento

-- ranking de avaliação da agência de publicidade - país de origem da montadora - potências dos veículos

1 -	Volvo Caminhões	BETC Havas	01/03/2014	1	Suécia	540 cv - 460 cv	
2 -	Volkswagen Caminhões	Galeria	23/01/2018	2	Alemanha	175 cv	
3 -	DAF	Galeria	18/08/2020	2	Holanda	530 cv	
4 -	Volvo Caminhões	BETC Havas	17/04/2014	1	Suécia	540 cv - 460 cv	

```
Filtros -- data mínima de lançamento: 01/01/2012 - potência mínima do veículo: 150
- ranking máximo de avaliação da agência de publicidade: 2 - país de origem da montadora: Suécia
Lançamentos : nome da montadora - nome da agência de publicidade - data do lançamento
-- ranking de avaliação da agência de publicidade - país de origem da montadora - potências dos veículos
1 - | Volvo Caminhões      | BETC Havas | 01/03/2014 | 1 | Suécia      | 540 cv - 460 cv |
2 - | Volvo Caminhões      | BETC Havas | 17/04/2014 | 1 | Suécia      | 540 cv - 460 cv |
```

3 - Estruturas de Dados Genéricas

Antes de concluir, é importante comentar sobre estruturas de dados genéricas no contexto da tipagem dinâmica de Python.

Linguagens orientadas a objeto, como Java e C++, tem tipagem estática. Ou seja, para declarar uma variável ou um parâmetro você precisa informar previamente o seu tipo, e para todo comando que altere o valor da variável ou que passe um argumento para o parâmetro, o compilador da linguagem irá checar se o valor atribuído pertence ao tipo previamente especificado.

Python é uma linguagem orientada a objetos com tipagem dinâmica. Ou seja, você não declara o tipo de uma variável ou parâmetro ao defini-lo. O tipo da variável ou parâmetro será inferido quando um valor for atribuído.

Com a tipagem dinâmica, você perde a capacidade do compilador de verificar se os valores atribuídos a variáveis ou parâmetros pertencem a um dado tipo, especificado previamente; então, você deve estar mais atento à implementação do seu código. Em contrapartida, você ganha muito em flexibilidade. Por exemplo, uma mesma função que soma dois números poderá ser executada para somar valores do tipo `int` ou `float`. Indo mais além, quando você define conjuntos de dados (listas e dicionários), você pode inserir objetos de classes distintas nesses conjuntos, ou até valores que não sejam objetos.

Em linguagens fortemente tipadas, as estruturas de dados são genéricas, porque podem ser definidas com base na classe, cujo objetos serão utilizados para povoar a estrutura de dados. De forma semelhante, existe o conceito de classe genérica que pode ser definida baseada em tipos em aberto (ex: `Lista<T>`) para utilização nas suas definições de dados e métodos. Antes de utilizar a classe genérica você precisará definir os tipos que haviam ficado em aberto (ex: `Lista<Veículo>`). A partir da mesma classe genérica, você poderá facilmente gerar novas classes simplesmente por atribuir diferentes tipos (ex: `Lista<AgênciaPublicidade>`).

Neste ponto, podemos constatar a versatilidade de linguagens com tipagem dinâmica, como Python. Você simplesmente não necessita definir previamente uma lista a partir de um dado tipo e nem uma classe genérica baseada em tipos em aberto. Simplesmente, os tipos já são naturalmente genéricos e serão inferidos, quando os valores forem atribuídos.

Neste ponto, finalizamos o Tutorial 2 com a ilustração completa da Etapa 2 do projeto de referência.